

Traducción del Alfabeto de la Lengua de Signos en Tiempo Real y Acercamiento a la Detección De Signos Dinámicos en Tiempo Real

Carlos García Jiménez

carlosgarciaj@correo.ugr.es

Younes Aghani

younesaghani@correo.ugr.es

Alonso Lucas Juberías

alonsolucasj@correo.ugr.es

Álvaro Gómez García

algoga@correo.ugr.es

Abstract

Hemos desarrollado varios modelos de reconocimiento de los signos del abecedario en LSE y un modelo de reconocimiento de signos dinámicos de hasta 64 clases distintas. El trabajo se ha centrado en encontrar la mejor forma de trabajar este problema y poder hacerlo a tiempo real. Principalmente se observa que los mejores resultados provienen de una detección de la posición de las principales partes de la mano para el posterior entrenamiento de los modelos, además de esto, nos encontramos con una gran falta de trabajo en este ámbito, dado que los datasets preparados para ello son muy escasos. Aún así los resultados son muy buenos y se puede mejorar mucho la tecnología disponible en torno a este problema.

1. Introduction

El problema a resolver se puede dividir en dos, por un lado la detección de signos estáticos, en este caso el abecedario en LSE (excluyendo la letra "Ñ" ya que incluye movimiento. Este problema, aunque por sí solo no es del todo útil, ya que la comunicación en LSE no se puede realizar únicamente signando letra a letra; aún así, nos sirve para dar los primeros pasos de cara a reconocer gestos y signos más complejos, entendiendo cómo deberían funcionar estos modelos y qué partes resultan más importantes a la hora de reconocer los gestos de las manos. Además, nos puede ayudar a comprender y a poner en práctica distintas formas de hacer el trabajo de maneras menos costosas a nivel computacional, o al menos, lo suficiente para que el reconocimiento se haga en tiempo real. Por tanto nuestro objetivo en esta parte es una detección lo suficientemente rápida y precisa para que se puedan observar los primeros pasos de cara a una comunicación clara. También vamos a comparar el rendimiento de distintos modelos para ampliar nuestro estudio sobre los mismos en la asignatura.

La segunda parte a resolver se centra en el re-

conocimiento automático de gestos dinámicos en el lenguaje de señas, abordando el desafío de procesar información proveniente de videos en lugar de imágenes estáticas. Esto resulta crucial porque la comunicación en señas depende significativamente del contexto y de la continuidad temporal, ya que muchos gestos incluyen movimientos complejos que no pueden ser interpretados correctamente desde un solo fotograma. Este proyecto no solo busca reconocer gestos individuales, sino también establecer las bases para sistemas que, en el futuro, puedan funcionar en tiempo real. Para alcanzar estos objetivos, se emplean redes neuronales convolucionales preentrenadas para extraer características espaciales de los fotogramas, combinadas con redes neuronales recurrentes (GRU) para modelar las relaciones temporales entre ellos.

2. Background

Para abordar el problema de gestos dinámicos se han utilizado redes neuronales recurrentes (RNN) que son un tipo de red neuronal diseñada para modelar secuencias y datos temporales. Estas redes tienen la capacidad de mantener un "estado interno", lo que les permite capturar dependencias a lo largo del tiempo. Sin embargo, las RNN tradicionales sufren problemas como el desvanecimiento del gradiente al trabajar con secuencias largas. Para mitigar esto, se introdujeron variantes avanzadas como LSTM (Long Short-Term Memory) y GRU (Gated Recurrent Units). GRU, una versión simplificada de LSTM, utiliza mecanismos como puertas de actualización y reinicio para controlar qué información retener y olvidar, lo que las hace computacionalmente más eficientes y adecuadas para tareas como el reconocimiento de gestos en videos. Además, procesar videos para entrenamiento es un desafío técnico porque los modelos deben capturar tanto las características espaciales de cada fotograma como las relaciones temporales entre ellos. Dividir los videos en fotogramas y procesarlos con redes como GRU, después de extraer características con modelos convolucionales preentrenados, permite abordar este prob-

lema, pero requiere una adecuada selección de parámetros y un diseño eficiente para manejar secuencias largas y complejas.

3. Related Works

Trabajando este problema se encuentran varios modelos y posibles soluciones (aunque menos de los que se esperaría viendo la utilidad que tiene y la necesidad de trabajo alrededor del mismo).

Como trabajo similar encontramos modelos como [10], tomando como referencia el mismo para partir del mismo dataset [4]. También se ha encontrado el modelo de red neuronal del que hemos partido en nuestros experimentos [1]. Estas investigaciones y experimentos tienen resultados en común que consisten en que son redes neuronales relativamente simples, con un bajo número de parámetros, pero presentan problemas parecidos. Según Ortiz-Farfán y Camargo-Mendoza, "Hoy en día no existe un repositorio público de imágenes o video que contenga estas señas ni la información necesaria para alcanzar esta meta, siendo uno de los principales impedimentos para iniciar la tarea" [10, p. 198]. En algunos se encuentran resultados óptimos en los conjuntos de entrenamiento y validación, y un rendimiento notablemente más bajo con nuevas imágenes, y en otros se han tenido que construir datasets propios y no se han obtenido resultados suficientemente buenos. Nos encontraremos con situaciones parecidas a lo largo del desarrollo de nuestro proyecto.

También se han encontrado investigaciones que comparan la extracción de características y clasificación con deep learning [13] que, basados en sus resultados, concluyen unos mejores resultados con el "approach" de deep learning.

La investigación en reconocimiento automático de gestos dinámicos ha evolucionado significativamente, desde métodos tradicionales basados en características manuales y clasificadores como HMMs o SVMs, hasta el uso de redes neuronales profundas que combinan capacidades espaciales y temporales. Los primeros enfoques carecían de escalabilidad y precisión en contextos complejos, lo que impulsó el desarrollo de arquitecturas híbridas que integran redes neuronales convolucionales (CNNs) para la extracción de características espaciales de fotogramas individuales, y redes neuronales recurrentes (RNNs), como LSTMs y GRUs, para modelar la continuidad temporal. Las GRUs, en particular, han ganado popularidad por ser más eficientes computacionalmente que las LSTMs, sin sacrificar rendimiento en tareas secuenciales. Estudios recientes destacan el uso de CNNs preentrenadas como ResNet y VGG, combinadas con RNNs, para abordar la naturaleza compleja de los videos. Además, los avances en arquitecturas modernas, como Transformers, prometen un modelo más eficiente de las relaciones espaciales y tempo-

rales. Sin embargo, desafíos como la latencia en el procesamiento en tiempo real y la necesidad de grandes bases de datos anotadas persisten. Conjuntos como [Chalearn LAP](#) [5] y [RWTH-PHOENIX-Weather](#) [7] han facilitado evaluaciones estandarizadas, pero aun existen brechas en la generalización y la diversidad cultural del lenguaje de señas.

4. Methods

En los experimentos trabajamos con varios modelos, los cuales hemos trabajado de distintas formas.

4.1. CNN Base

En primer lugar, partimos de un modelo [1] con pocos parámetros (se desglosarán más adelante) que ha demostrado dar buenos resultados en los conjuntos del dataset de MNIST [4] para el alfabeto americano en lengua de signos. Queremos ver la capacidad de reutilizar el mismo para nuestros datasets [8, 14] usando fine-tuning.

4.1.1 Data pre-processing

El preprocesamiento que se usa es muy simple, comenzando por redimensionar las imágenes a 28x28, pasándolas a escala de grises y guardándolas en vectores de una sola dimensión. Posteriormente, se han normalizado los valores de las intensidades de la escala de grises.

4.1.2 Arquitectura del modelo

Para empezar la arquitectura del modelo base está compuesto de una capa convolucional que aplica 30 filtros de 5×5 , que contribuye a un total de $(5 \times 5 \times 1 + 1) \times 30 = 780$ parámetros, una segunda capa convolucional de 15 filtros de 3×3 sobre la entrada que contribuye con $(3 \times 3 \times 30 + 1) \times 15 = 4065$ parámetros, una capa de maxpooling después de cada capa convolucional y una capa Dropout antes de las capas densas que ayuda a mejorar la actuación del modelo [10].

En cuanto a las capas densas tenemos una capa de tamaño de entrada 375 y tamaño de salida 128 que contiene $(375 + 1) \times 128 = 48128$ parámetros, una capa densa de tamaño de salida 50 que contribuye con $(128 + 1) \times 50 = 6450$ parámetros al modelo y por último una capa densa de tamaño 26 de salida (las clases que tiene que reconocer) que aporta $(50 + 1) \times 26 = 1326$ parámetros. Además detrás de cada capa convolucional y densa tenemos una capa de activación ReLU que se encarga de introducir no linealidad en el modelo, permitiendo que las activaciones de los filtros sean positivas y maximicen la información útil que la red aprende sobre los patrones de las imágenes. Esto cambia en la última capa densa la cuál usa una capa de activación Softmax, ya que estamos trabajando con un problema de clasificación multiclase. Por lo que en total tenemos 60749

parámetros en nuestro modelo, concentrándose en su gran mayoría en las capas densas.

4.2. Modelos Clásicos

- Random Forest (RandomForestClassifier): Es un ensamblaje de árboles de decisión que se construyen de forma independiente utilizando muestras aleatorias del conjunto de entrenamiento y características. El resultado final se obtiene por votación o promedio de los árboles. Es robusto ante el sobreajuste y maneja bien datos con ruido y relaciones no lineales.
- Regresión Logística (LogisticRegression): Utiliza una función sigmoide para estimar la probabilidad de que una observación pertenezca a una clase. Es comúnmente usado para clasificación binaria y puede ser extendido a multiclase. Se emplea con el parámetro `max_iter=1000` para asegurar la convergencia en problemas complejos.
- K-Nearest Neighbors (KNeighborsClassifier): Clasifica una muestra según los k vecinos más cercanos en el espacio de características, calculando la distancia entre ellos. Es simple, pero puede ser costoso computacionalmente, especialmente con grandes volúmenes de datos.
- XGBoost (XGBClassifier): Basado en Gradient Boosting, mejora los modelos débiles (generalmente árboles de decisión) secuencialmente, corrigiendo errores del modelo anterior. Es eficiente y eficaz en tareas de aprendizaje automático, con parámetros como `use_label_encoder=False` y `eval_metric='logloss'` para optimización.
- Árbol de Decisión (DecisionTreeClassifier): Divide el espacio de características en regiones según reglas de los datos, generando un árbol que predice la clase en cada hoja. Es fácil de interpretar, pero puede sobreajustarse si no se poda adecuadamente.

4.3. Yolov4-tiny

Para este modelo se ha partido del modelo Yolov4-tiny [2].

4.3.1 Arquitectura del modelo

Este modelo se basa en la arquitectura YOLO que fue propuesta originalmente por Joseph Redmon [12], y posteriormente Alexey Bochkovskiy [3] hizo un estudio y implemento una versión mejor del modelo que es la que se ha utilizado en este proyecto. La arquitectura original se basa en GoogleNet, en total tiene 137 capas convolucionales, cuatro capas de maxpooling y dos capas totalmente conectadas. En cambio, el modelo implementado tiene 29 capas

convolucionales, con función de activación leaky ReLU, y no tiene capas totalmente conectadas. Por el contrario, tiene dos capas de salida yolo. Cada capa de salida procesa características a una resolución diferente, derivadas de una parte específica del mapa de características, para conseguir detectar objetos de diferentes tamaños con precisión.

Este modelo está entre los detectores de una etapa, que tienen una gran velocidad para detectar objetos acompañado de una alta precisión. Esto es especialmente relevante si vamos a querer que el modelo funcione correctamente en pruebas de tiempo real. La arquitectura YOLO funciona mejor que otras arquitecturas de detección de objetos [6] pero, a la vez, tiene varias implementaciones en código abierto lo cual facilita mucho trabajar con él.

4.4. Detección de gestos dinámicos

El enfoque propuesto combina técnicas de extracción de características espaciales y modelado temporal para reconocer gestos dinámicos en videos. Para la extracción de características espaciales, se utiliza ResNet50, una red convolucional profunda preentrenada en ImageNet, excluyendo su capa final de clasificación para obtener un vector compacto que represente patrones visuales relevantes. Los pesos de ResNet50 se congelan durante el entrenamiento, aprovechando su capacidad para detectar características genéricas como bordes, texturas y formas, lo que reduce la carga computacional y mejora la eficiencia.

El modelado temporal se realiza mediante una red GRU bidireccional, que mejora la capacidad del modelo para capturar dependencias tanto hacia adelante como hacia atrás en la secuencia de gestos. Las características espaciales extraídas de los fotogramas se procesan en secuencia, y el modelo utiliza máscaras para manejar datos de longitudes variables, asegurando que solo los fotogramas válidos contribuyan al aprendizaje.

ResNet50 es ampliamente utilizada en problemas de visión por computadora debido a su capacidad para aprender representaciones visuales genéricas que pueden transferirse eficientemente a tareas específicas.

El diseño del modelo incluye una función de pérdida adecuada para clasificación multiclase y un optimizador basado en gradiente que ajusta los pesos del modelo. Esta combinación permite capturar tanto patrones espaciales como temporales en los gestos dinámicos, proporcionando una solución robusta para este problema.

5. Experimentos

5.1. CNN y Modelos Clásicos

5.1.1 Dataset

La preparación del dataset inicial ha consistido principalmente en el uso de dos conjuntos de datos:

- "Spanish Sign Language Alphabet (Static)" [8]: Este conjunto de datos contiene imágenes con fondo blanco de tamaño 4288x2408 píxeles para cada letra del alfabeto español (salvo la "Ñ"), incluyendo aquellas letras que requieren gestos dinámicos. En este trabajo, filtramos los datos para usar únicamente las letras con gestos estáticos.
- "Lenguaje de Signos Español" [14]: Este conjunto de datos contiene imágenes con fondo azul de tamaño 230x250 píxeles, correspondientes a las letras que requieren gestos estáticos. No fue necesario realizar filtrado en este conjunto.

Ambos datasets están disponibles públicamente en Kaggle. Cada conjunto contiene alrededor de 100 imágenes por letra, distribuidas equitativamente entre todas las letras, lo que representa una base sólida para traducir los signos de la mano al alfabeto español.

En este trabajo se necesitan dos datasets ya que se pretende comparar diferentes enfoques para la detección y traducción del gesto.

La creación del primer dataset se ha realizado almacenando las imágenes del conjunto inicial pero en escala de grises, redimensionándolas a 28x28 píxeles y normalizando los valores de intensidad.

La creación del segundo dataset se ha hecho recorriendo las imágenes del conjunto inicial, y almacenar para cada imagen los valores de los puntos de referencia detectados por Mediapipe. Dichos valores han sido normalizados de manera que dependan de los dos puntos de referencia con el valor más pequeño en el eje horizontal y con el valor más pequeño en el eje vertical. Esta normalización permite que los valores obtenidos no dependan de la posición de la mano en el frame ni de la distancia que la separa de la cámara.

Además, como primer conjunto de entrenamiento de la red neuronal, se ha utilizado el dataset "Sign Language MNIST", también presente en Kaggle.

5.1.2 Entrenamiento

La red neuronal convolucional se ha entrenado inicialmente con el dataset "Sign Language MNIST". Se ha compilado usando la función de pérdida "sparse_categorical_crossentropy", el optimizador "Adam" y la métrica "accuracy". El entrenamiento se ha realizado durante 7 épocas, utilizando un 20% de los datos de entrenamiento como conjunto de validación. Finalmente, el modelo ha sido evaluado en el conjunto de prueba, mostrando la pérdida y el accuracy alcanzados, lo que permite medir su desempeño en la tarea de clasificación.

Para convertir las etiquetas de texto en valores numéricos procesables por los modelos implementados, se han codificado mediante LabelEncoder de scikit-learn. El dataset

se ha dividido en un conjunto de entrenamiento (80%) y otro de prueba (20%). Los modelos basados en extracción de características han sido: Random Forest Classifier, Logistic Regression, KNeighbors Classifier, XGBoost Classifier y Decision Tree Classifier. Mientras tanto, la red neuronal obtenida previamente ha sido entrenada nuevamente, pero con el dataset de la lengua de signos española. Antes de dicho entrenamiento, se han congelado todas las capas de la red neuronal, excepto las densas (Dense), y se ha vuelto a compilar el modelo de la misma manera (sparse_categorical_crossentropy, Adam, accuracy), especificando una tasa de aprendizaje baja ($5e-3$). Con esto se ha obtenido un modelo que ha sido entrenado durante 10 épocas, utilizando el mismo porcentaje de datos de validación que los modelos clásicos (20%).

5.1.3 Resultados

La métrica principal utilizada para la comparación del rendimiento de los modelos ha sido el accuracy score (Figura 1.). Además, la matriz de confusión (Figura 2.) ha sido un buen indicador de la cantidad de predicciones acertadas por cada modelo.

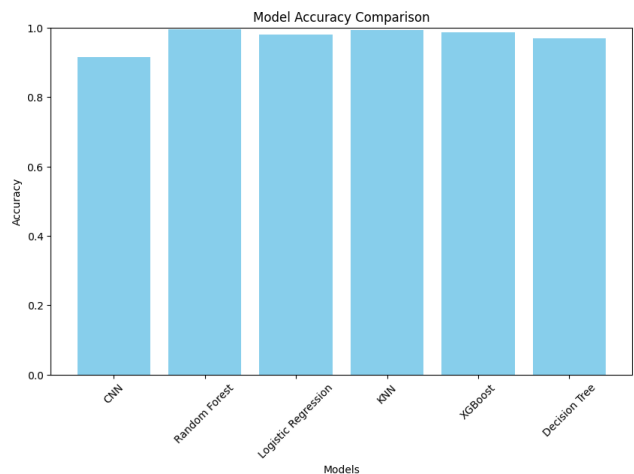


Figure 1. Model Accuracy Comparison

Se ha observado que todos los modelos presentan un accuracy que supera el 90%. Además, los modelos clásicos parecen ser más eficaces globalmente comparados a la red neuronal convolucional, destacándose el modelo basado en Random Forest con un valor muy cercano al 100%.

En cuanto a las matrices de confusión, generalmente las predicciones son correctas para todas las letras, pero existen algunas confusiones en el caso de la red neuronal, que se pueden observar por ejemplo con las letras "M" y "N", al tener signos representativos bastante similares. En esta visualización, también se muestra que el Random Forest es el modelo que mejor predice las etiquetas.

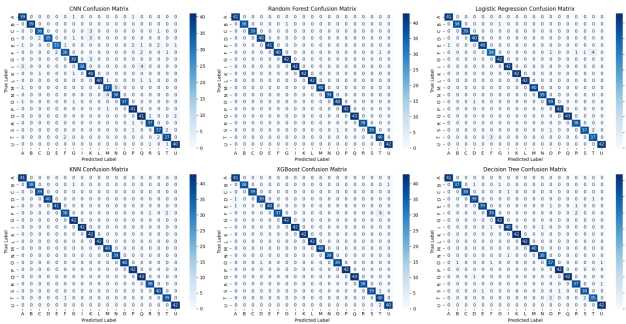


Figure 2. Model Confusion Matrix Comparison

5.1.4 Data Augmentation

Se ha utilizado la técnica de "data augmentation" para aumentar la diversidad del conjunto de datos de entrenamiento y prueba, con el objetivo de mejorar la generalización del modelo y mitigar el sobreajuste. Esta técnica genera nuevas muestras a partir de las existentes mediante la aplicación de transformaciones aleatorias, lo que permite al modelo aprender características invariantes a dichas transformaciones.

Se definió un ratio de augmentación del 50%, lo que implica que la mitad del conjunto de datos original fue seleccionada aleatoriamente para aplicar las transformaciones. Para esto, se calculó el número de muestras a aumentar tanto para los conjuntos de entrenamiento como de prueba, y se seleccionaron subconjuntos aleatorios de cada uno utilizando una función de muestreo aleatorio sin reemplazo.

El pipeline de augmentación incluyó las siguientes transformaciones implementadas mediante capas de augmentación de TensorFlow:

- Rotación aleatoria: rotaciones en un rango de $[-10^\circ, +10^\circ]$.
- Traslación aleatoria: desplazamientos horizontales y verticales de hasta el 10% de la imagen.
- Zoom aleatorio: ampliaciones o reducciones de hasta el 30%.
- Contraste aleatorio: variaciones en el contraste de hasta un 30%.
- Ruido gaussiano: adición de ruido con una desviación estándar de 0.4.

Estas transformaciones fueron aplicadas tanto a los conjuntos de entrenamiento como de prueba seleccionados.

Posteriormente, las muestras aumentadas fueron combinadas con los datos originales para formar nuevos conjuntos de entrenamiento y prueba ampliados. Las dimensiones de los conjuntos resultantes fueron verificadas para garantizar la correcta combinación de datos.

El modelo inicial, una red neuronal convolucional básica (CNN), fue entrenado nuevamente utilizando los conjuntos aumentados. Los valores y parámetros utilizados fueron similares al caso anterior. Se obtuvo un valor de accuracy de un 65%, lo que representa un considerable empeoramiento en comparación con los modelos anteriores.

5.1.5 Pruebas

Para el reconocimiento de gestos manuales, se implementó un sistema en tiempo real utilizando la biblioteca MediaPipe y un modelo de clasificación basado en Random Forest. El sistema captura imágenes desde una cámara web mediante OpenCV, procesando cada cuadro para detectar y extraer características de las manos. Para la detección de manos, se utilizó el modelo MediaPipe Hands, configurado en modo dinámico con un umbral de confianza mínimo de 0.3 para la detección. Una vez identificadas las manos, se obtienen los 21 puntos clave de la mano, representados como coordenadas normalizadas (x, y). Estas coordenadas fueron normalizadas dentro de un marco delimitador que ajusta la escala y posición de los puntos clave para garantizar invariancia a traslaciones y escalas. Posteriormente, las características extraídas se organizan en un vector y se ingresaron al modelo de clasificación Random Forest previamente entrenado, el cual predice la clase del gesto correspondiente. Además, se dibuja un cuadro delimitador alrededor de la mano detectada y se superpone el carácter predicho como texto en el video en tiempo real. Este proceso se repite de manera continua, logrando una inferencia en tiempo real mientras el sistema mantiene un flujo constante de cuadros desde la cámara. La implementación incluye también mecanismos para detener la ejecución mediante la tecla ESC.

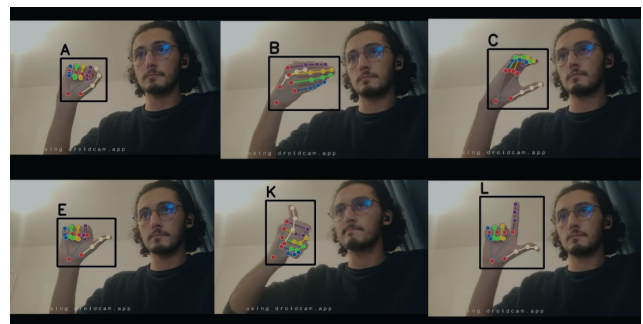


Figure 3. Ejemplos de uso de Random Forest en tiempo real

5.2. Yolov4-tiny para el alfabeto americano

5.2.1 Dataset

El dataset usado para el entrenamiento y los test del modelo está compuesto por imágenes de signos de la lengua de sig-

nos americana (ASL). 1512 imágenes para el entrenamiento del modelo, 144 para la validación y 72 para el test. Las imágenes están acompañadas de ficheros de anotación con formato xml. En estos ficheros se indica la ubicación de los objetos en forma de recuadros delimitadores y son necesarios para entrenar el modelo. Las imágenes están divididas en 26 clases, cada una es una letra del abecedario americano.

5.2.2 Entrenamiento

Para entrenar el modelo, en primer lugar se ha descargado el modelo preentrenado [2] y se ha entrenado con nuestro dataset, para el entrenamiento el modelo YOLOv4-tiny utiliza una función de pérdida compuesta para optimizar la detección de objetos. Esta función combina tres aspectos:

Pérdida de coordenadas: Penaliza las diferencias entre las coordenadas predichas y las reales de las cajas delimitadoras que acompañan a las imágenes. Pérdida de confianza: Optimiza la predicción de la probabilidad de que una caja contenga un objeto. Pérdida de clasificación: Evalúa la precisión en la clasificación del objeto detectado en una de las 26 clases.

Para la función de optimización Darknet utiliza SGD (Stochastic Gradient Descent) como el optimizador por defecto. Se ajustan sus parámetros en los ficheros de control. Algunos de los parámetros que se ajustan son la tasa de aprendizaje (que decae a intervalos que también se ajustan), el momento (para mejorar la estabilidad del gradiente durante el entrenamiento) o regularizar los pesos para evitar el sobreajuste.

5.2.3 Resultados

Para medir los resultados del test se utilizan varias métricas diferentes como mAP (precisión promedio), IoU (intersección sobre unión), Precision, Recall y F1-Score.

Según métricas como la mAP del 0.8678 o el Recall del 0.88 parece indicar que el modelo detecta la mayoría de los objetos y tiene un buen rendimiento en general. El IoU Promedio = 65.60% de lo cual se puede deducir que las cajas delimitadoras predichas están razonablemente cerca de las cajas reales.

La precisión es del 0.79, lo cual es relativamente baja. Si nos fijamos en los positivos o negativos totales vemos que los falsos positivos totales son relativamente grandes, TP (true positive)= 63, FP (false positive) = 17, FN (false negative)= 9. Destacan las dos clases con ap = 0.00% (E y L), es decir, el modelo no ha aprendido a detectar esas clases en concreto.

Así pues, el modelo tiene un rendimiento sólido. Sin embargo, la precisión no es muy alta. No atermiando de discernir entre las clases. Habría que trabajar en intentar

mejorar la detección de algunas clases para así aumentar la precisión.

Posteriormente se ha implementado el código para poder probar el modelo en tiempo real:

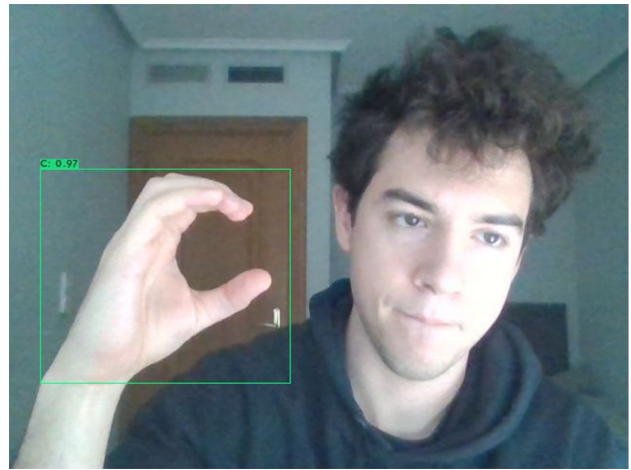


Figure 4. Ejemplo de uso del modelo en tiempo real con la detección correcta de la letra "C".

5.3. Detección de gestos dinámicos

5.3.1 Dataset

El reconocimiento de la Lengua de Señas se abordó utilizando el dataset LSA64 [11], un conjunto de datos ampliamente reconocido que contiene 3200 videos de 64 señas diferentes. Específicamente, se utilizó la versión disponible en Kaggle [9], que incluye etiquetas organizadas en un archivo CSV y proporciona una partición inicial de entrenamiento (2560 videos) y prueba (640 videos), uniformemente distribuidos con 40 videos por clase en entrenamiento y 10 en prueba.

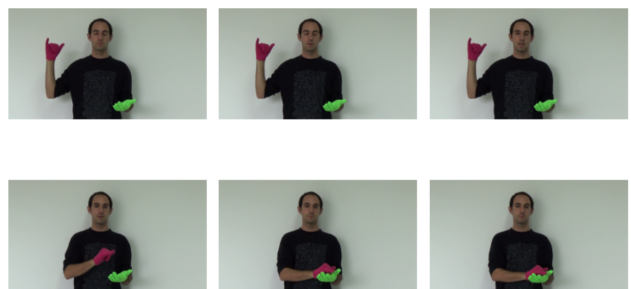


Figure 5. Ejemplo de fotogramas extraídos de un video del dataset LSA64.

El dataset LSA64 presenta varios aspectos que lo hacen interesante pero desafiante para el reconocimiento automático de gestos. Los videos tienen un fondo simple y

uniforme, y el uso de guantes de colores brillantes ayuda a resaltar las manos que realizan los gestos. Sin embargo, algunos gestos involucran movimientos complejos que incluyen ambas manos, lo que requiere un modelo capaz de capturar relaciones espaciales y temporales con precisión. Además, las variaciones en iluminación y los detalles específicos de los dedos añaden complejidad al problema. En general, el modelo debe ser robusto ante estas variaciones mientras capta la dinámica temporal de los gestos.

5.3.2 Data Partitioning

El conjunto de entrenamiento se dividió en un subconjunto de validación (20%), estratificando por clases para mantener una distribución proporcional. Esto resultó en 2048 videos para entrenamiento y 512 para validación. Esta separación previa a la extracción de características evita el fenómeno de *data snooping*, asegurando una evaluación imparcial y representativa del modelo.

5.3.3 Data Augmentation

Para mejorar la robustez del modelo, se implementaron técnicas de *data augmentation* en el conjunto de entrenamiento. Estas incluyeron ajustes aleatorios de brillo, contraste, desplazamientos, *zoom*, y ruido gaussiano, simulando condiciones de grabación variables. Estas estrategias incrementaron la diversidad de los datos sin requerir más videos, ayudando al modelo a manejar variaciones de iluminación, posición, y calidad de grabación.

5.3.4 Feature and Sequence Processing

Cada video fue representado como una secuencia de características extraídas a partir de fotogramas equidistantes. Estos fotogramas se recortaron al centro, redimensionaron a 128x128 píxeles y procesaron utilizando ResNet50, configurada como extractor de características. La red generó vectores compactos que encapsulan patrones visuales relevantes, y los pesos de ResNet50 permanecieron congelados para reducir la carga computacional.

Para manejar videos con longitudes variables, se generaron máscaras que identifican cuáles fotogramas dentro de una secuencia son válidos, basándose en un límite definido y usando para todos 20 fotogramas. Estas secuencias, junto con las etiquetas numéricas producidas por un procesador de etiquetas, se prepararon como entradas para el modelo recurrente.

5.3.5 Model and Training Configuration

El modelo propuesto utiliza una red GRU bidireccional con 128 unidades. La entrada incluye las características y las

máscaras, que identifican los fotogramas válidos. Las salidas de la GRU se condensan mediante *GlobalAveragePooling1D* y pasan a una capa densa de 128 unidades con activación ReLU. Una capa *Dropout* con una tasa del (30%) reduce el sobreajuste, y la capa de salida con activación softmax produce probabilidades para las 64 clases. El modelo tiene 1.7 millones de parámetros entrenables y se compila utilizando la pérdida categórica, el optimizador Adam y la métrica de precisión. Para el entrenamiento, se configuró un máximo de 50 épocas con lotes de tamaño 32 y *EarlyStopping* detiene el proceso si no había mejoras en la pérdida de validación durante 10 épocas consecutivas. Este diseño asegura una clasificación precisa y eficiente de los gestos dinámicos en diversas condiciones.

5.3.6 Results

Curvas de entrenamiento y validación: La Figura 6 presenta las curvas de precisión y pérdida para los conjuntos de entrenamiento y validación a lo largo de las épocas. Se observa que ambos conjuntos alcanzan una precisión superior al 90%, con una convergencia estable y una pérdida que disminuye progresivamente en ambos casos. Estos resultados iniciales sugieren que el modelo logra capturar patrones relevantes en los datos de entrenamiento y generaliza bien durante la validación.

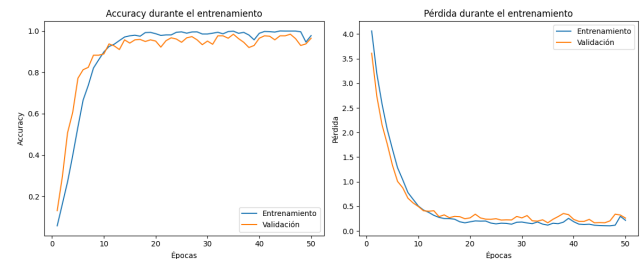


Figure 6. Curvas de precisión y pérdida durante el entrenamiento y la validación.

Resultados en el conjunto de prueba: La evaluación en el conjunto de prueba arroja una precisión del (72.19%), significativamente menor en comparación con el desempeño en entrenamiento y validación (ambos superiores al 90%).

Matriz de confusión en el conjunto de prueba: La Figura 7 muestra la matriz de confusión, donde se evidencia que el modelo identifica correctamente la mayoría de las clases, pero presenta confusiones notables en gestos similares.

6. Conclusions

En el experimento sobre la detección del alfabeto español, se obtuvieron resultados concluyentes que desta-

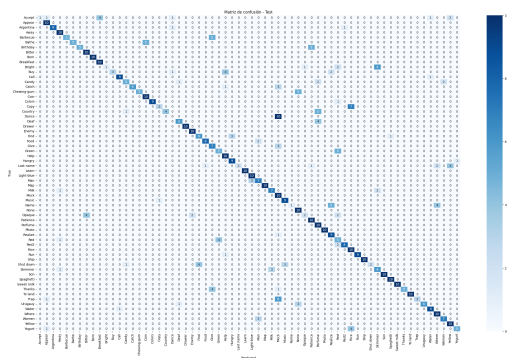


Figure 7. Matriz de confusión para el conjunto de prueba.

can las diferencias de rendimiento entre los modelos clásicos y el modelo CNN inicial:

- Los modelos clásicos como Random Forest son particularmente efectivos en problemas donde las características relevantes ya han sido preprocesadas y definidas de forma explícita, como en este caso, con la normalización y extracción de puntos clave. Estos modelos son menos propensos a sobreajustarse y suelen generalizar mejor en datasets pequeños o medianos.
- Aunque la red convolucional es capaz de aprender características complejas, su arquitectura básica (con solo dos capas convolucionales y menos de 61,000 parámetros) puede no ser suficiente para capturar la variabilidad de los gestos, especialmente en un conjunto de datos limitado. Esto, sumado a posibles confusiones en gestos similares (como se observó con las letras "M" y "N"), afecta su desempeño.
- El experimento se ha visto muy influenciado por el dataset, con el cual hemos visto que al ser tan limitado ha provocado que la red neuronal convolucional no haya alcanzado el potencial que podría haber alcanzado con un dataset de mayor cantidad y calidad.

Todo esto resalta la importancia de seleccionar modelos adecuados al tamaño y características del conjunto de datos. Además de la posibilidad que hay de alcanzar resultados óptimos entorno a este problema.

En cuanto al desarrollo de este modelo para el reconocimiento de gestos en lenguaje de señas las conclusiones que podemos extraer son que ha sido una experiencia que refleja tanto las fortalezas como los desafíos inherentes al problema. El modelo, basado en redes neuronales recurrentes bidireccionales y apoyado por un extractor de características preentrenado como ResNet50, mostró un excelente rendimiento en los conjuntos de entrenamiento y validación, pero una caída en el conjunto de prueba, alcanzando

una exactitud del 0.7219. Aunque el conjunto de datos estaba uniformemente distribuido en términos de clases, la cantidad total de ejemplos por clase sigue siendo limitada, lo que podría dificultar al modelo capturar la variabilidad completa de los gestos. La brecha entre los resultados de validación y prueba podría atribuirse a ligeras diferencias en las condiciones de captura de los videos, lo que subraya la importancia de trabajar con datos diversos y representativos. Sin embargo, este trabajo sienta una base sólida y demuestra el potencial de las redes neuronales recurrentes para abordar problemas complejos como este, mostrando un camino claro para futuras mejoras en la calidad y diversidad de los datos o en la optimización de la arquitectura del modelo.

References

- [1] Namas Bhandari. Sign language mnist cnn, 2019. GitHub repository. 2
- [2] Alexey Bochkovskiy. darknet, 2020. 3, 6
- [3] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan M. Liao. YOLOv4: Optimal speed and accuracy of object detection, 2020. 3
- [4] datamunge. Sign language mnist dataset, 2018. Kaggle dataset. 2
- [5] Sergio Escalera, Vassilis Athitsos, and Isabelle Guyon. Chalearn looking at people challenge 2014: Dataset and results. In *European conference on computer vision*, pages 459–473. Springer, 2014. 2
- [6] Snehal Joshi. Top object detection models, 2024. Online. 3
- [7] Oscar Koller, Jens Forster, and Hermann Ney. Continuous sign language recognition: Towards large vocabulary statistical recognition systems handling multiple signers. *Computer Vision and Image Understanding*, 141:108–125, 2015. 2
- [8] Kirle Lea. Spanish sign language alphabet (static), 2023. Kaggle dataset. 2, 4
- [9] Marc Marais. Lsa64 dataset on kaggle, 2020. 6
- [10] Nelson Ortiz-Farfán and Jorge E. Camargo-Mendoza. Modelo computacional para reconocimiento de lenguaje de señas en un contexto colombiano. *TecnoLógicas*, 23:197–232, 2020. 2
- [11] Facundo Quiroga. Lsa64: A dataset for argentinian sign language. 2020. 6
- [12] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection, 2016. 3
- [13] R. Sreemathy, Mousami Turuk, Isha Kulkarni, and Soumya Khurana. Sign language recognition using artificial intelligence. *Education and Information Technologies*, 28:5259–5278, 2023. 2
- [14] Rubén Tercero. Lenguaje de signos español, 2024. Kaggle dataset. 2, 4